# DA5402: MLOps Engineering

Final Project Report

NYC Taxi Demand Forecasting & Driver Dispatch Optimization

Rishwanth P B (ED21B052)

April 22, 2026

## Contents

# 1 Problem Statement

Urban taxi fleets are deployed reactively rather than proactively. Dispatchers allocate drivers based on intuition and static rules, causing two simultaneous failure modes: driver shortages in high-demand zones where passengers face long waits, and idle driver oversupply in low-demand zones where drivers waste fuel and time.

The root cause is the absence of a reliable, zone-level, forward-looking demand forecast. This project addresses the problem by building a production-grade, MLOps-driven system that predicts hourly taxi pickup demand per NYC zone 6 hours into the future, enabling intelligent pre-positioning of drivers before demand materializes.

# 2 Dataset

## 2.1 Source

The dataset is sourced from the New York City Taxi and Limousine Commission (TLC), published as monthly Parquet files at `nyc.gov/tlc`. The data is completely free with no sign-up required.

## 2.2 Data Selection

Selected months from 2019, 2020, and 2023 are downloaded and merged to form a training corpus of approximately 40–50 million records spanning 263 NYC taxi zones.

Table 1: Dataset Selection Rationale

| Period | Files | Purpose |
|---|---|---|
| 2019 (all 12 months) | 12 files | Pre-COVID baseline — training data |
| 2020 April | 1 file | COVID collapse — primary drift trigger |
| 2023 Jan–Apr | 4 files | Post-COVID new normal — retraining target |
| **Total** | **17 files** | **≈40–50M records** |

## 2.3 Key Columns

Table 2: Dataset Columns and Usage

| Column | Type | Role |
|---|---|---|
| `tpep_pickup_datetime` | Timestamp | Feature engineering, target derivation |
| `PULocationID` | Integer (1–263) | Primary grouping key |
| `trip_distance` | Float | Zone-level trip type proxy |
| `passenger_count` | Float | Zone occupancy signal |
| `RatecodeID` | Integer | Airport zone identification |
| `congestion_surcharge` | Float | Manhattan core zone flag |
| `airport_fee` | Float | Dropped — null in 2019 data |

### 2.4 Target Variable

The target variable is engineered directly from the raw data:

Listing 1: Target variable derivation

```
demand = COUNT(*)
         GROUP BY PULocationID, HOUR(tpep_pickup_datetime)
```

Each row in the feature dataset represents one zone, one hour, with the number of taxi pickups in that zone during that hour.

## 3 System Architecture

The system follows a strict loose-coupling principle between the frontend and backend, connected only via configurable REST API calls.

Table 3: System Architecture Layers

| Layer | Technology | Responsibility |
|---|---|---|
| Data Ingestion | Apache Spark + Airflow | Raw Parquet ingestion, cleaning, feature engineering |
| Versioning | DVC + Git + Git LFS | Data, model, and code versioning |
| Experiments | MLflow | Training tracking, model registry |
| Serving | FastAPI + LightGBM | REST API for real-time inference |
| Frontend | Streamlit | Dispatcher dashboard |
| Monitoring | Prometheus + Grafana | Metrics, drift detection, alerting |
| Deployment | Docker Compose | Container orchestration |

## 4 MLOps Implementation

### 4.1 Data Engineering Pipeline

Apache Spark processes the raw monthly Parquet files, orchestrated by an Apache Airflow DAG. The pipeline runs in two stages per file.

**Stage 1 — Cleaning (`spark_clean.py`):**

- Filters records to valid date range per month
- Drops unused columns: `airport_fee`, `store_and_fwd_flag`
- Fills null `congestion_surcharge` values with 0
- Drops rows with null `passenger_count` or `RatecodeID`

**Stage 2 — Feature Engineering (`spark_features.py`):**

Table 4: Engineered Features

| Feature | Derived From | Purpose |
|---|---|---|
| `demand` | COUNT(*) per zone per hour | Target variable |
| `hour_of_day` | HOUR(pickup_datetime) | Intra-day patterns |
| `day_of_week` | DAYOFWEEK(pickup_datetime) | Weekday vs weekend |
| `month` | MONTH(pickup_datetime) | Seasonal patterns |
| `is_weekend` | day_of_week IN (1,7) | Weekend demand shift |
| `is_rush_hour` | hour IN (7,8,9,17,18,19) | Peak commuter hours |
| `demand_lag_1h` | Lag 1 per zone | Short-term autocorrelation |
| `demand_lag_24h` | Lag 24 per zone | Same hour yesterday |
| `demand_lag_168h` | Lag 168 per zone | Same hour last week |
| `rolling_mean_24h` | 24-hour rolling avg | Short-term trend |
| `rolling_mean_7d` | 168-hour rolling avg | Medium-term baseline |
| `is_airport_zone` | PULocationID IN (1,132,138) | Airport zone flag |

Throughput: one monthly file (3–5M records) processed in under 3 minutes on a 3-worker local Spark cluster.

## 4.2 Source Control and Continuous Integration

- **Git**: All application code, DAG definitions, Dockerfiles, and MLflow project files are version-controlled.
- **Git LFS**: Raw monthly Parquet files tracked with Git LFS, keeping the main repository lightweight.
- **DVC**: The complete pipeline from raw data to trained model is defined as a DVC DAG. `dvc repro` reproduces the full pipeline from scratch. Every experiment is uniquely identified by a Git commit hash paired with an MLflow Run ID.

## 4.3 Experiment Tracking

All model training runs are tracked in MLflow. Each run logs:

- Model type and all hyperparameters from `config.yaml`
- Evaluation metrics: RMSE, MAE, MAPE, $R^2$
- Feature importance rankings (top 10 by SHAP value)
- DVC dataset hash for full reproducibility
- Training duration and memory usage

Baseline statistics (mean, variance, distribution of hourly demand per zone) are computed during EDA and logged as MLflow artifacts for later use in drift detection.

## 4.4 Prometheus Instrumentation

Five custom metrics are exposed at the `/metrics` endpoint:

Table 5: Prometheus Metrics

| Metric | Type | Description |
|---|---|---|
| `prediction_requests_total` | Counter | Total predictions by endpoint and zone |
| `prediction_latency_seconds` | Histogram | Prediction latency distribution |
| `model_confidence_score` | Gauge | Current model confidence (0–1) |
| `api_error_total` | Counter | Total API errors by endpoint |
| `demand_drift_score` | Gauge | KL divergence from training baseline |

Three Grafana dashboards monitor these metrics in near real-time:

1. **Operations Dashboard**: request rate, P95 latency, error rate
2. **Forecast Quality Dashboard**: MAPE trend, drift score per zone group
3. **Pipeline Health Dashboard**: Airflow DAG timeline, Spark job durations

Alert rules fire when:

- `demand_drift_score` $> 0.15$ (KL divergence threshold)
- `api_error_rate` $> 5\%$
- Prediction latency P95 $> 200$ms

### 4.5 Software Packaging and Deployment

- **MLflow**: Production model saved natively using `booster_.save_model()` for fast loading without pickle overhead.
- **FastAPI**: Wraps the LightGBM inference with business logic for demand classification and pre-positioning recommendations. Implements `/health` and `/ready` probes.
- **Docker Compose**: Three services — backend, Prometheus, Grafana — running as isolated containers on a shared Docker network.
- **MLprojects**: `MLproject` file with `conda.yaml` defines the training environment precisely, ensuring identical dev and production environments.

## 5 Model Development

### 5.1 Train / Validation / Test Split

A strict temporal split is used — random splits are never used for time-series data as they cause data leakage.

Table 6: Data Split

| Split | Months | Rows | Purpose |
|---|---|---|---|
| Train | All 2019 | 716,086 | Model training |
| Validation | 2023 Jan–Feb | 76,919 | Hyperparameter tuning |
| Test | 2023 Mar–Apr + 2020 Apr | 104,054 | Final evaluation + drift test |

## 5.2 Models Trained

Four models were trained and compared in MLflow:

Table 7: Model Comparison Results

| Model | Val RMSE | Val MAE | Val MAPE | Val $R^2$ |
|---|---|---|---|---|
| XGBoost | 18.60 | 10.27 | 62.71% | 0.9467 |
| LightGBM | 18.35 | 10.10 | 60.10% | 0.9481 |
| LSTM | 20.39 | 11.05 | 58.72% | 0.9360 |
| TFT | — | **5.07** | — | — |

## 5.3 Model Selection Rationale

**TFT (Temporal Fusion Transformer)** achieves the best MAE of 5.07 — roughly half the error of tree-based models — because it uses attention mechanisms to learn complex temporal dependencies directly from sequences. TFT is used for **batch forecasting**.

**LightGBM** is used for **real-time REST inference**. TFT requires 168 hours of sequential context per zone to make a prediction and cannot serve single-row REST API requests. LightGBM takes one row with pre-computed lag features and returns a prediction in milliseconds.

**LSTM underperforms** tree models because the lag features already explicitly capture temporal patterns. Tree models use these engineered features directly and efficiently. LSTM would benefit from raw sequences without pre-engineered lags.

# 6 REST API

The FastAPI backend exposes the following endpoints:

Table 8: API Endpoints

| Method | Endpoint | Description |
|---|---|---|
| GET | `/health` | API liveness check |
| GET | `/ready` | Model loaded and ready check |
| POST | `/predict` | Single zone demand prediction |
| POST | `/predict/batch` | Multi-zone batch prediction |
| POST | `/recommend` | Driver pre-positioning recommendation |
| POST | `/update-drift` | Recompute KL divergence drift score |
| GET | `/metrics` | Prometheus metrics endpoint |
| GET | `/docs` | Auto-generated API documentation |

### 6.1 Demand Classification Logic

Each prediction is classified into one of three operational states:

- **Surge**: predicted/rolling_mean $> 1.5$
- **Dead Zone**: predicted/rolling_mean $< 0.3$
- **Normal**: all other cases

### 6.2 Driver Recommendation Logic

Listing 2: Driver recommendation rule

```
recommended_drivers = max(1, int(predicted_demand / 15))
diff = recommended_drivers - current_drivers

if diff > 2:
    action = f"Send␣{diff}␣more␣drivers␣to␣this␣zone"
elif diff < -2:
    action = f"Relocate␣{abs(diff)}␣drivers␣from␣this␣zone"
else:
    action = "Driver␣count␣is␣optimal"
```

## 7 Drift Detection and Automated Retraining

### 7.1 Drift Detection

KL divergence is used to measure distributional shift between live zone demand and the 2019 baseline:

$$D_{KL}(P\|Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

where $P$ is the live demand distribution and $Q$ is the 2019 baseline distribution. A score near 0 indicates no drift; a high score indicates the live distribution has shifted significantly from baseline.

## 7.2 Drift Results

Table 9: KL Divergence Drift Scores

| **Period** | **Drift Score** | **Interpretation** |
|---|---|---|
| 2019-01 (baseline vs itself) | 0.0077 | No drift — correct |
| 2023-04 (current production) | 0.0326 | Low drift — no retraining needed |
| 2020-04 (COVID collapse) | 0.7695 | High drift — retraining triggered |
| **Alert threshold** | **0.15** | Retraining triggers above this |

The 2020 April score of 0.7695 is five times the alert threshold, confirming that the COVID-19 demand collapse is clearly detectable as drift.

## 7.3 Automated Retraining Pipeline

The Airflow DAG `retraining_pipeline` runs daily with four tasks:

1. `check_drift` — calls `/update-drift` API, checks if score $> 0.15$
2. `retrain_model` — runs `train_lightgbm.py` using the active virtual environment Python
3. `validate_model` — verifies the retrained model file exists and is accessible
4. `reset_drift_score` — calls `/update-drift` to refresh the Prometheus metric after retraining

All four tasks were verified as successful (green) in the Airflow UI.

# 8 Frontend Dashboard

The Streamlit dashboard provides four pages for dispatch operators:

Table 10: Dashboard Pages

| Page | Description |
|---|---|
| Dashboard | KPI cards + bar chart of top 10 zone demand forecasts with surge/dead zone color coding |
| Single Zone Prediction | Interactive form to predict demand for any zone with custom inputs |
| Zone Heatmap | Folium map of NYC with demand circles per zone — red = Surge, green = Normal, blue = Dead Zone |
| Driver Recommendations | Input current drivers and predicted demand to get recommended action |

The frontend calls the FastAPI backend exclusively via REST API calls. The API base URL is an environment variable — the two services have zero compile-time dependency on each other.

# 9 Testing

## 9.1 Unit Tests

Six unit tests were written using `pytest` and FastAPI's `TestClient`:

Table 11: Test Cases and Results

| Test | Assertion | Result |
|---|---|---|
| `test_health` | Status 200, `status=ok` | PASS |
| `test_ready` | Status 200, `status=ready` | PASS |
| `test_predict` | Status 200, `predicted_demand > 0` | PASS |
| `test_predict_surge` | Valid `demand_status` value | PASS |
| `test_recommend` | `recommended_drivers > 0` | PASS |
| `test_predict_batch` | 2 predictions returned for 2 zones | PASS |
| **Total** | | **6/6 PASS** |

Listing 3: Running the test suite

```
python -m pytest tests/test_api.py -v
# 6 passed in 2.24s
```

# 10 Problems Faced and Solutions

Table 12: Problems and Solutions

| Problem | Solution |
|---|---|
| LightGBM pickle loading via MLflow hung indefinitely | Saved model natively using `booster_.save_model()` and loaded with `lgb.Booster(model_file=path)` |
| Airflow retraining task used system Python instead of venv Python | Used `sys.executable` in subprocess call to always point to the active interpreter |
| TFT predictions had device mismatch (MPS vs CPU) | Moved model to CPU with `model.cpu()` before running predictions; manually extracted outputs and targets to CPU tensors |
| Docker port 8000 occupied by WordPress container from another assignment | Identified via `docker ps` and stopped the conflicting container before running our compose stack |
| Airflow webserver port 8080 occupied by assignment-6 container | Stopped `assignment-6-chefwork24-airflow-webserver-1` before starting our local Airflow instance |
| Drift endpoint returning 404 from Dockerized backend | Docker image was built before the endpoint was added; switched to running the backend locally during demo |

# 11 Results Summary

## 11.1 ML Metrics

Table 13: Final Model Performance (LightGBM — Production Model)

| Metric | Validation | Test |
|---|---|---|
| RMSE | 18.35 | 16.48 |
| MAE | 10.10 | 8.58 |
| MAPE | 60.10% | 62.28% |
| $R^2$ | 0.9481 | 0.9556 |

The high MAPE is driven by low-demand zones where even small absolute errors create large percentage errors. $R^2$ of 0.95 means the model explains 95% of demand variance — the primary metric for this use case.

### 11.2 System Metrics

Table 14: System Performance

| Metric | Value |
|---|---|
| Prediction latency P95 | $< 200$ms |
| Drift score (2023 production data) | 0.0326 (below 0.15 threshold) |
| Drift score (2020 COVID data) | 0.7695 (five times threshold) |
| Unit tests passing | 6/6 |
| API endpoints | 7 |
| Docker services | 3 (backend, Prometheus, Grafana) |
| Airflow DAG tasks (all green) | 4 |

## 12 Conclusion

A complete, end-to-end MLOps project was built for NYC taxi demand forecasting and driver dispatch optimization. The key outcomes are:

- **40–50 million records** processed via Apache Spark across 17 monthly Parquet files, orchestrated by Airflow, versioned by DVC
- **4 models** trained and compared in MLflow — TFT achieved the best MAE of 5.07, LightGBM used for real-time inference with $R^2$ of 0.95
- **7 FastAPI endpoints** with Pydantic validation, Prometheus instrumentation, and 6/6 unit tests passing
- **Streamlit dashboard** with 4 pages — demand heatmap, zone predictions, driver recommendations, and operations view
- **Full MLOps loop closed** — drift detection via KL divergence triggers automated Airflow retraining when distribution shift exceeds 0.15 threshold, demonstrated with COVID-19 collapse scoring 0.77
- **3-container Docker Compose** stack with Prometheus and Grafana monitoring, 3-panel operations dashboard, and alert rules for drift, latency, and error rate